SmartCanteen - Digital Queue & Payment Management System

A full-stack web application for college canteens to manage food ordering, digital payments, and queue management.

Features

For Students

- Browse menu with categories (Snacks, Drinks, Meals)
- Add items to cart and place orders
- Pay online using UPI, Card, or Wallet
- Track order status in real-time
- View order history

For Admin (Canteen Staff)

- Dashboard with order statistics and revenue
- Manage menu items (add, edit, delete, toggle availability)
- View and manage orders
- Update order status (Preparing → Ready → Completed)
- Real-time queue display

Technical Features

- Real-time updates using WebSocket (Socket.io)
- JWT-based authentication
- Role-based access control
- Clean Architecture with layered design
- OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction)
- Design Patterns (Factory, Strategy, Observer)
- SOLID principles

Tech Stack

Backend

- Node.js with Express.js
- TypeScript
- MongoDB (Mongoose)
- JWT for authentication
- bcrypt for password hashing
- Socket.io for real-time updates

Frontend

- Next.js 14 (App Router)
- React 18
- TypeScript
- Tailwind CSS
- Socket.io-client

Project Structure

```
smart-canteen/
├── README.md
├── backend/
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts          # Backend entry point
│       ├── config/           # Database and app configuration
│       ├── controllers/      # HTTP request handlers
│       ├── db/               # Mongoose schemas/models
│       ├── interfaces/       # TypeScript interfaces/contracts
│       ├── middleware/       # Auth and error handling
│       ├── models/           # OOP User classes
│       ├── patterns/         # Design patterns (Factory, Strategy, Observer)
│       ├── repositories/     # Data access layer
│       ├── routes/           # API routes
│       ├── services/         # Business logic
│       └── websocket/        # Socket.io handler
│
├── docs/
│   └── uml/                  # UML diagrams (use case, ER, sequence)
│
├── frontend/
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.js
│   ├── app/                  # Next.js app router pages
│   │   ├── admin/          # Admin dashboard & management
│   │   ├── cart/           # Shopping cart
│   │   ├── dashboard/      # Student dashboard
│   │   ├── login/          # Login page
│   │   ├── menu/           # Menu page
│   │   ├── orders/         # Order history
│   │   ├── queue/          # Queue display
│   │   └── signup/         # Registration
│   ├── components/         # React components
│   │   ├── layout/         # Navbar, etc.
│   │   ├── menu/           # Menu components
│   │   ├── order/          # Order components
│   │   ├── queue/          # Queue display
│   │   └── ui/             # Reusable UI components
│   ├── contexts/           # React contexts (Auth, Cart)
│   ├── services/           # API and Socket services
```

```
│   ├── types/              # TypeScript types
│   └── lib/                # Utilities
│
├── database/
│   └── schema.sql          # Legacy SQL schema (optional reference)
├── package.json            # Root scripts (workspace-level)
└── package-lock.json
```

Setup Instructions

Prerequisites

- Node.js 18+
- MongoDB 6+
- npm or yarn

1. Database Setup

```
# Start local MongoDB (example with Homebrew services)
brew services start mongodb-community
```

2. Backend Setup

```
cd backend

# Install dependencies
npm install

# Create .env file
cp .env.example .env

# Edit .env with your database credentials
# MONGODB_URI=mongodb://127.0.0.1:27017/smart_canteen
# JWT_SECRET=your-secret-key
# PORT=5000

# Setup database indexes
npm run db:setup

# Seed sample data
npm run db:seed

# Start development server
npm run dev
```

3. Frontend Setup

```
cd frontend

# Install dependencies
npm install

# Create .env.local (optional)
echo "NEXT_PUBLIC_API_URL=http://localhost:5000/api" > .env.local
echo "NEXT_PUBLIC_SOCKET_URL=http://localhost:5000" >> .env.local

# Start development server
npm run dev
```

4. Access the Application

- Frontend: http://localhost:3000
- Backend API: http://localhost:5000/api
- Queue Display: http://localhost:3000/queue

Demo Credentials

After running the seed script:

- **Admin:** admin@smartcanteen.com / admin123
- **Student:** john@student.edu / student123

API Endpoints

Authentication

- `POST /api/auth/signup` - Register new user
- `POST /api/auth/login` - Login user
- `GET /api/auth/profile` - Get current user profile
- `PUT /api/auth/profile` - Update profile
- `GET /api/auth/verify` - Verify token

Menu

- `GET /api/menu` - Get all menu items
- `GET /api/menu/:id` - Get menu item by ID
- `GET /api/menu/category/:category` - Get items by category
- `GET /api/menu/grouped` - Get menu grouped by category
- `GET /api/menu/search?q=term` - Search menu

Orders

- `POST /api/orders` - Create new order
- `GET /api/orders` - Get user's orders
- `GET /api/orders/:id` - Get order by ID
- `DELETE /api/orders/:id` - Cancel order

Payments

- `GET /api/payments/methods` - Get payment methods
- `POST /api/payments` - Process payment
- `GET /api/payments/order/:orderId` - Get payment by order

Queue

- `GET /api/queue` - Get current queue status

Admin

- `GET /api/admin/dashboard` - Get dashboard stats
- `POST /api/admin/menu` - Create menu item
- `PUT /api/admin/menu/:id` - Update menu item
- `DELETE /api/admin/menu/:id` - Delete menu item
- `PATCH /api/admin/menu/:id/toggle` - Toggle availability
- `GET /api/admin/orders` - Get all orders
- `GET /api/admin/orders/active` - Get active orders
- `PATCH /api/admin/orders/:id/status` - Update order status

WebSocket Events

Client to Server

- `authenticate` - Authenticate with JWT token
- `join:queue` - Join queue display room
- `join:admin` - Join admin room
- `subscribe:order` - Subscribe to order updates
- `unsubscribe:order` - Unsubscribe from order

Server to Client

- `queue:update` - Single queue item update
- `queue:full` - Full queue state
- `order:status` - Order status update
- `order:ready` - Order ready notification
- `admin:new-order` - New order notification
- `admin:order-update` - Order update for admins

Design Patterns Used

1. Factory Pattern (UserFactory)

Creates `Student` or `Admin` objects from common user data based on role.

- **Where:** `backend/src/patterns/UserFactory.ts`, used in `backend/src/services/AuthService.ts`
- **Why:** Centralizes role-based object creation and avoids repeated `if/else` role checks.

2. Strategy Pattern (PaymentStrategy)

Interchangeable payment methods (UPI, Card, Wallet) selected at runtime.

- **Where:** `backend/src/patterns/PaymentStrategy.ts`, used in `backend/src/services/PaymentService.ts`
- **Why:** Each method has different validation/processing, and new methods can be added without changing service logic.

3. Observer Pattern (QueueObserver)

Real-time queue updates where one subject (queue manager) notifies many observers (socket clients).

- **Where:** `backend/src/patterns/QueueObserver.ts`, integrated in `backend/src/websocket/socketHandler.ts`
- **Why:** Enables one-to-many push updates for queue and order state changes.

4. Singleton Pattern (QueueManager + shared instances)

Ensures a single shared instance for stateful components.

- **Where (strict singleton):** `QueueManager.getInstance()` in `backend/src/patterns/QueueObserver.ts`
- **Where (singleton-style exports):** `authService`, `orderService`, `paymentService`, repositories, `paymentContext`, `userFactory`
- **Why:** Queue state must be globally consistent in-process, and shared service/repository instances keep wiring simple.

SOLID Principles

- **S** - Single Responsibility: Separate services (AuthService, OrderService, PaymentService)
- **O** - Open/Closed: Add new payment methods without modifying existing code
- **L** - Liskov Substitution: Student/Admin interchangeable as User
- **I** - Interface Segregation: Separate interfaces for API contracts, repositories, queue observers, and payment strategies
- **D** - Dependency Inversion: Services are constructed with repository dependencies and can be injected/swapped for testing

Building for Production

Backend

```
cd backend
npm run build
npm start
```

Frontend

```
cd frontend
npm run build
npm start
```

License

MIT License